Reflection homework 13a

Do Huy Du - 202417108

May 23, 2026

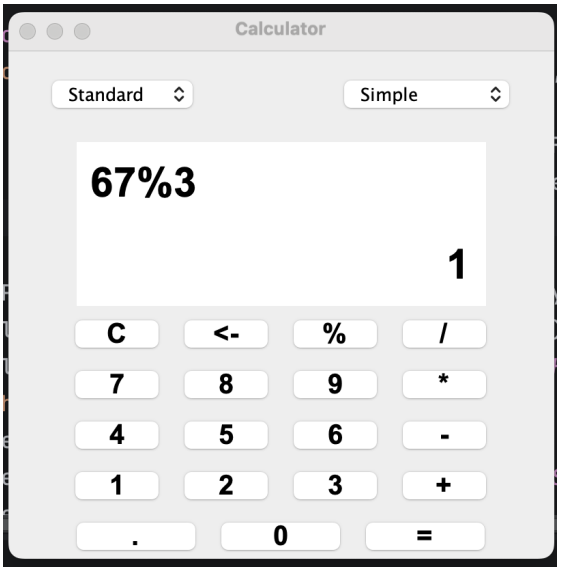
1 Design decisions

I decided to separate the UI logic (Calculator.java) from event handling (ButtonListener.java) and calculation logic (Evaluator, Parser, Lexer) to keep the application maintainable. I also let users type in the full expression and evaluate it only when they press "=" rather than recalculating on every keystroke.

For the expression evaluation algorithm, I chose the Shunting Yard algorithm, which I had learned from the DSA course. Lastly, to handle overflow and underflow, I used the `java.math.BigDecimal` library.

2 Main functionalities

My inspiration came from the Casio fx-580, so my calculator has two display areas — one for the input expression and one for the result. It has digit buttons (0–9), a decimal point, and currently supports the "+", "-", "*", "/", and "%" operators. The calculator also has control buttons: "C" (clear all), "←" (delete last character), and "=" to evaluate. To feel like a real calculator, I also programmed it to handle exceptions such as division by zero and incomplete or invalid expressions.



3 Implementation overview

I treated the user's input expression as a sequence of `Token` objects — a class I created to represent both operands and operators. The sequence is split into a `List<Token>` by the Lexer, then the infix expression is handed to the Parser, which converts it into postfix form. Finally, the Evaluator takes that postfix expression and returns the result.

Inside `ButtonListener`, this whole pipeline is wrapped inside the event handler for the "=" button. For all other buttons, `ButtonListener` simply updates the input display as normal.

4 Challenges

The challenge did not come from handling edge cases. The hardest part was building the calculation logic itself — implementing the Shunting Yard algorithm — since I had to go back and revisit what I had learned from the DSA course.